Learned Rescue Routing for Decoder Portfolios in a Structured Quantum Error-Correction Benchmark

Arham Khalid
Department of Physics, IIT Delhi

June 1, 2026

Abstract

Quantum error correction requires decoders that infer logical corrections from noisy syndrome data. Different decoders often embody different inductive biases: matching-based decoders exploit graph structure, lookup decoders exploit repeated patterns, heuristic decoders exploit low-weight cases, and neural decoders exploit learned nonlinear correlations. This motivates a meta-decoding question: can a learned router select between a portfolio of decoders and outperform the best individual decoder?

We study this question in a structured, quantum-error-correction-inspired syndrome-history benchmark designed to exhibit heterogeneous error regimes and decoder complementarity. The portfolio contains a cheap heuristic decoder, an MWPM-like repetition decoder, a union-find-inspired proxy, an empirical maximum-likelihood-style lookup decoder, an MLP decoder, and a CNN decoder. Initial multiclass router, expected-utility, residual-rescue, and specialist-routing approaches either failed to beat the best decoder or did not scale. Diagnostics showed a separation between oracle complementarity and learnable complementarity: although an oracle portfolio achieved very high accuracy, naive routers could not reliably infer which decoder would be correct.

The final successful method is an accuracy-only rescue router. It uses the strongest individual decoder as a default and learns rare override gates for candidate decoders. To avoid overfitting to decoder behavior on the training split, the gate models are trained on one held-out validation subset and their thresholds are tuned on a separate held-out subset. On the final structured benchmark, the best individual decoder was a CNN with accuracy 0.762687. The learned full rescue router achieved accuracy 0.765312, a gain of +0.002625. Bootstrap resampling over 16,000 test samples gave a 95 percent confidence interval of [+0.000375,+0.004938] and probability of positive gain equal to 0.9864. An MLP-only rescue ablation achieved accuracy 0.765875, indicating that most of the learned gain came from rare CNN-to-MLP rescue switches. Random-switch controls with the same switch fraction were consistently worse than the baseline. These results support the modest but meaningful claim that learned rescue routing can recover a statistically significant part of decoder complementarity in a controlled heterogeneous QEC-inspired setting.

Contents

1 Introduction

Fault-tolerant quantum computation requires quantum error correction (QEC), where repeated syndrome measurements are used to infer and correct physical errors before they accumulate into logical failures. A central component of any QEC protocol is the decoder: a classical algorithm that maps observed syndrome information to a correction or logical prediction.

Many decoders have been developed for topological and stabilizer codes. Minimum-weight perfect matching (MWPM) is a widely used decoding strategy for surface-code-like settings and related graph-structured syndrome problems [1, 2]. Union-find decoders offer faster alternatives for certain topological-code settings [4]. Neural decoders can learn nonlinear features from data but may require significant training and can be sensitive to the distribution of training examples. Lookup or empirical maximum-likelihood decoders can be effective on familiar low-distance or repeated patterns, but may fail outside their memorized support.

These differences motivate the hypothesis of decoder complementarity: no single decoder is uniformly best across all syndrome regimes, and a portfolio of decoders may contain more information than any one decoder alone. If this is true, then a learned meta-decoder or router may be able to outperform the strongest individual decoder by selecting the right decoder for each sample.

This project investigates that hypothesis in a controlled QEC-inspired setting. The central question is: can a learned router exploit decoder complementarity to outperform the best individual decoder? The answer is subtle. Oracle complementarity is easy to demonstrate: if we are allowed to inspect the true outcome and pick whichever decoder is correct, the portfolio can achieve extremely high accuracy. The hard part is learnable complementarity: can a router infer from observable syndrome and decoder metadata when a non-default decoder should be trusted?

The final result of this project is a small but statistically meaningful learned-router gain over the best individual decoder on a structured heterogeneous benchmark. Equally important, the path to this result involved several failed routers and diagnostic redesigns. These failures are included because they reveal the distinction between apparent oracle headroom and actually learnable routing structure.

2 Background and Motivation

2.1 Decoder portfolios

A decoder portfolio is a set of candidate decoders

$$\mathcal{D} = \{D_1, D_2, \ldots, D_K\},$$

where each decoder maps a syndrome history X to a predicted logical label:

$$\hat{y}_k = D_k(X).$$

In this project, the portfolio contains six decoders:

1. **Cheap heuristic decoder**: a low-compute rule-based decoder using simple syndrome-weight statistics.
2. **MWPM-like repetition decoder**: a repetition-code-inspired matching proxy based on smoothed syndrome chains.
3. **Union-find proxy decoder**: a cluster-geometry heuristic inspired by union-find decoding.
4. **Empirical MLD small decoder**: a lookup-style empirical maximum-likelihood decoder.
5. **MLP neural decoder**: a fully connected neural network operating on flattened syndrome histories and summary features.
6. **CNN neural decoder**: a convolutional neural network operating directly on syndrome-history arrays.

The goal is not to claim these are state-of-the-art QEC decoders. Rather, the portfolio is designed to contain different inductive biases so that routing can be studied in a controlled way.

2.2 Oracle complementarity

For a sample i, decoder D_k is correct if

$$\mathbf{1}[\hat{y}_{ik} = y_i] = 1.$$

The best individual decoder has accuracy

$$A_{\text{best}} = \max_k \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\hat{y}_{ik} = y_i].$$

The all-decoder oracle accuracy is

$$A_{\text{oracle}} = \frac{1}{N} \sum_{i=1}^{N} \max_k \mathbf{1}[\hat{y}_{ik} = y_i].$$

The oracle gain is $A_{\text{oracle}} - A_{\text{best}}$.

A large oracle gain indicates that different decoders are correct on different samples. However, this does not by itself prove that routing is learnable. The oracle is allowed to use the true label. A learned router must choose a decoder using only observable information available at inference time.

2.3 Learned routing

A learned router is a function

$$R(X, m_1, \ldots, m_K) \in \{1, \ldots, K\},$$

where m_k denotes optional metadata from decoder D_k, such as confidence values, chain weights, cluster scores, lookup support, or internal diagnostic features. The routed prediction is

$$\hat{y}_R = D_{R(X)}(X).$$

The central challenge is that the router must predict decoder reliability, not the true label directly.

3 Benchmark Design

3.1 Syndrome-history representation

Each sample is represented as a small spacetime syndrome-history array

$$X \in \{0,1\}^{T\times(d-1)},$$

where T is the number of syndrome rounds and d is the effective code distance. The target label $y \in \{0,1\}$ is a logical-class proxy generated by a structured rule family.

This benchmark is QEC-inspired rather than a full circuit-level surface-code simulation. The purpose is to create a controlled environment where heterogeneous syndrome regimes and decoder complementarity can be studied before moving to more realistic Stim and PyMatching-style simulations [3, 2].

3.2 Structured regimes

The final benchmark uses a harder structured complementarity design. It includes several hidden regimes:

1. `easy_low_weight`: sparse or obvious low-weight and final-round patterns.
2. `iid_smooth`: smoother repetition-like syndrome chains.
3. `bursty_cluster`: spatially and temporally clustered syndrome activity.
4. `lookup_pattern`: repeated pattern families intended to favor empirical lookup behavior.
5. `nonlinear_mixed`: nonlinear combinations of global syndrome statistics.
6. `ambiguous_mixed`: deliberately ambiguous mixtures of smooth and bursty regimes.

Earlier benchmark versions were either too easy, allowing the CNN to reach roughly 98 percent accuracy, or too noisy and unstructured, preventing any learned router from finding reliable switches. The final benchmark was adjusted to produce a harder baseline while preserving a large oracle gap.

3.3 Scope of the benchmark

This benchmark is not claimed to be a complete physical surface-code simulation. It is a structured testbed for studying meta-decoding. A future version should replace the synthetic syndrome generator with circuit-level detector events sampled from tools such as Stim and decoded using realistic baselines such as PyMatching [3, 2]. The present result should therefore be interpreted as a proof-of-concept for learned rescue routing under heterogeneous QEC-inspired noise, not as a state-of-the-art QEC decoding result.

4 Decoder Portfolio

4.1 Cheap heuristic decoder

The cheap heuristic decoder uses simple syndrome statistics such as total syndrome weight, first-round weight, last-round weight, and threshold rules. It is computationally inexpensive and useful as a low-cost baseline, but it is not expected to be highly accurate on complex samples.

4.2 MWPM-like repetition decoder

The MWPM-like decoder smooths the syndrome history over time and constructs a minimal chain proxy. This captures the broad inductive bias of matching-based decoders: find a low-weight error configuration consistent with the observed syndrome. It is not a full Blossom or PyMatching implementation, but it serves as a structured graph-like baseline.

4.3 Union-find proxy decoder

The union-find proxy computes cluster geometry features such as cluster count, largest cluster size, spatial span, temporal span, persistent columns, and edge-touching behavior. It is inspired by the fact that union-find decoders exploit connected components and cluster growth in syndrome graphs [4]. In this project it is a proxy rather than a complete union-find decoder.

4.4 Empirical MLD small decoder

The empirical MLD decoder is a lookup-style decoder. It stores empirical pattern-label associations and can be strong on repeated or familiar patterns. However, this decoder also exposed an important methodological issue: training-split rescue rates could be misleading because lookup behavior can overfit to previously seen patterns.

4.5 MLP and CNN neural decoders

The neural decoders are trained supervised models. The MLP operates on flattened syndrome histories and summary features, while the CNN uses the two-dimensional syndrome-history structure. In the final benchmark, the CNN was the strongest individual decoder, but the MLP became the most useful rescue decoder.

5 Initial Routing Attempts and Failures

A major part of the project was discovering which routing formulations were insufficient.

5.1 Naive selector routing

The earliest learned routers attempted to predict which decoder should be selected directly. These multiclass selector routers performed poorly. They often switched too frequently and accumulated harmful switches. The main lesson was that choosing a decoder is harder than predicting the label: a router must understand comparative decoder reliability.

5.2 Expected-utility routing

Expected-utility routers attempted to trade off correctness and compute. Although utility-aware routing is interesting, it confounded the immediate goal. A low-cost decoder can be utility-optimal even when it is not accuracy-optimal. For this project, we eventually separated the objectives and focused on pure accuracy.

5.3 Residual rescue routing

Residual rescue routers attempted to default to the best decoder and switch only on predicted failure cases. On small runs, a switch-gated residual router briefly produced a tiny positive gain, but the gain did not survive scaling. This indicated that the early benchmark produced oracle complementarity but not robust learnable complementarity.

5.4 Specialist-regime routing

A later approach introduced explicit specialist regimes. A true-specialist diagnostic router showed whether routing would help if the correct hidden specialist were known. Initially, this helped, but on larger runs the CNN became too dominant and the specialist labels became misaligned with actual performance. This revealed another important point: specialist metadata is useful only if the named specialist remains genuinely competitive in that regime.

5.5 Diagnosis

The failures led to the following diagnosis:

> A large oracle gap is not enough. The router must be trained and validated on held-out decoder behavior, and the benchmark must contain failure modes that are both complementary and learnable.

This motivated the final accuracy-only rescue formulation.

6 Final Method: Accuracy-Only Rescue Routing

6.1 Baseline decoder

The final method uses the strongest individual decoder as the default. In the final run, this was the CNN decoder:

$$D_{\text{base}} = D_{\text{CNN}}.$$

The baseline prediction is used unless a candidate decoder passes a learned rescue gate.

6.2 Rescue and harm targets

For each candidate decoder D_c, two binary events are defined:

$$\text{rescue}_c = \mathbf{1}[D_{\text{base}}(X) \neq y \text{ and } D_c(X) = y],$$

$$\text{harm}_c = \mathbf{1}[D_{\text{base}}(X) = y \text{ and } D_c(X) \neq y].$$

A useful switch is one where rescue occurs. A harmful switch is one where harm occurs. The router should switch only when rescue probability exceeds harm probability by enough margin.

6.3 Held-out gate training

A key methodological improvement was to avoid training the rescue gate on decoder outputs from the same split used to train the decoders. The final protocol was:

1. Train the base decoders on the training split.
2. Evaluate all decoders on validation and test splits.
3. Split the validation set into two internal subsets: gate-train and gate-tune.
4. Train rescue and harm classifiers on gate-train.
5. Tune switching thresholds on gate-tune.
6. Evaluate the final router once on the test set.

This fixed a major issue with the empirical MLD decoder. On the original training split, empirical MLD appeared to have very high rescue rate and almost no harm, but this did not generalize. The held-out protocol prevented the router from trusting non-generalizable rescue signals.

6.4 Router score

For each candidate decoder, the router computes

$$s_c(X) = \hat{p}_{\text{rescue},c}(X) - \hat{p}_{\text{harm},c}(X),$$

where both probabilities are estimated by tabular ExtraTrees classifiers.

A switch to candidate c is allowed only if

$$s_c(X) \geq \tau_c,$$

where τ_c is tuned on the held-out gate-tuning split. If multiple candidates pass their thresholds, the router selects the candidate with the largest score. If no candidate passes, the router stays with the baseline CNN.

6.5 Features

The rescue router uses only test-time-available features:

- raw flattened syndrome bits,
- syndrome summary statistics,
- decoder predictions,
- decoder confidence values,
- pairwise decoder agreement and disagreement,
- baseline-vs-candidate prediction disagreement,
- baseline-vs-candidate confidence gaps,
- decoder metadata such as cluster scores, chain weights, lookup support, and related internal diagnostics.

The router does not use the true label or decoder correctness at test time.

7 Final Results

7.1 Main accuracy result

The final test set contained 16,000 samples. The best individual decoder was the CNN decoder with accuracy 0.762687. The all-decoder oracle achieved accuracy 0.995875, indicating substantial complementarity in the decoder portfolio.

The learned full accuracy-rescue router achieved accuracy 0.765312, improving over the CNN baseline by +0.002625. Bootstrap resampling gave a positive 95 percent confidence interval.

Table 1: Main final result.

Method	Accuracy	Gain over CNN	Notes
CNN baseline	0.762687	0.000000	Best individual decoder
Full accuracy-rescue router	0.765312	+0.002625	Learned router
Oracle accuracy router	0.995875	+0.233187	Upper bound only

7.2 Bootstrap significance

The observed gain of +0.002625 corresponds to 42 additional correct samples out of 16,000. Bootstrap resampling showed that this gain was unlikely to be a random fluctuation.

Table 2: Bootstrap significance of full rescue-router gain.

Quantity	Value
Test size	16,000
Baseline accuracy	0.762688
Router accuracy	0.765312
Observed gain	+0.002625
95 percent bootstrap CI	[+0.000375, +0.004938]
Probability of positive gain	0.9864

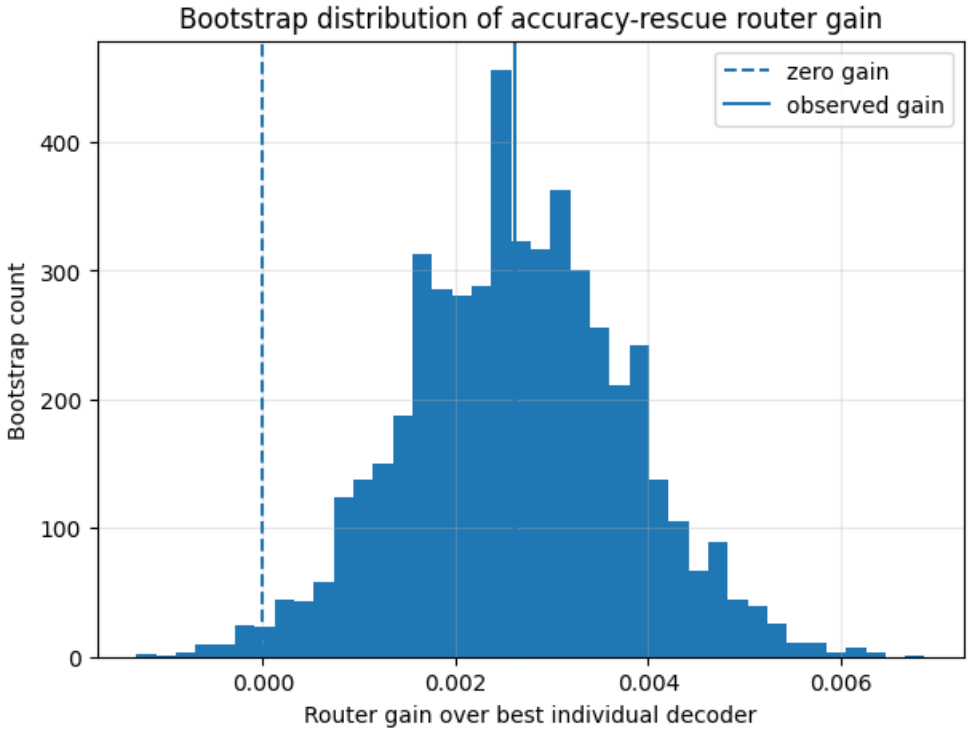

Figure 1: Bootstrap distribution of the accuracy-rescue router gain over the best individual decoder. The dashed vertical line marks zero gain, while the solid vertical line marks the observed gain.

7.3 Switch diagnostics

The router switched away from the CNN baseline on only 2.1 percent of test samples. Among 336 total switches, 189 were helpful and 147 were harmful. Thus, the net gain was 42 samples.

Table 3: Switch summary for the full accuracy-rescue router.

Quantity	Value
Baseline decoder	CNN neural decoder
Switch fraction	0.021
Total switches	336
Helpful switches	189
Harmful switches	147
Net correct samples	+42
Net accuracy gain	+0.002625

This is an important result: the router does not improve by broadly replacing the CNN. Instead, it makes rare override decisions where helpful switches slightly but consistently outnumber harmful switches.

7.4 Candidate-level switch breakdown

Most of the gain came from switching from CNN to the MLP decoder. Other decoders contributed smaller positive gains, while the cheap heuristic was slightly harmful in the final full-router breakdown.

Table 4: Switch breakdown by candidate decoder.

Candidate	Switches	Helpful	Harmful	Net samples	Net gain
MLP neural decoder	221	128	93	+35	+0.002188
MWPM-like repetition	6	5	1	+4	+0.000250
Empirical MLD small	69	36	33	+3	+0.000187
Union-find proxy	34	18	16	+2	+0.000125
Cheap heuristic	6	2	4	-2	-0.000125

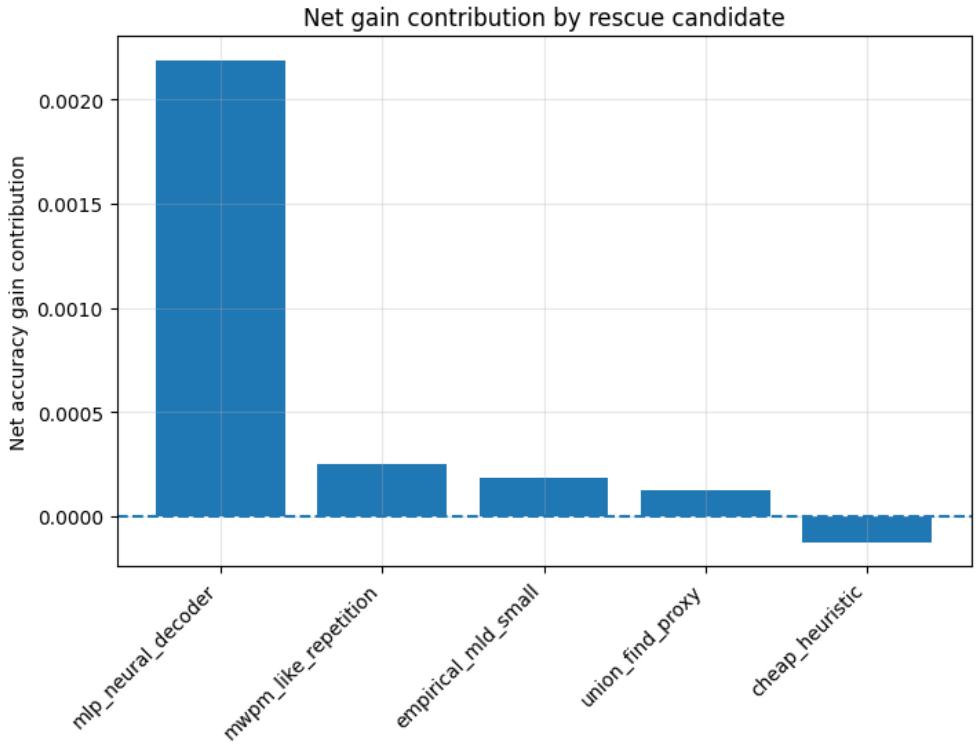

Figure 2: Net gain contribution by rescue candidate. The MLP decoder provides the dominant positive contribution, showing that the main learned mechanism is a rare CNN-to-MLP rescue rule.

The main learned mechanism is therefore best described as a CNN-to-MLP rescue rule, with smaller contributions from other specialists.

8 Ablations and Sanity Checks

8.1 Single-candidate rescue ablations

To understand which candidate decoders were responsible for the gain, single-candidate rescue ablations were evaluated. The MLP-only rescue router achieved the best learned-router accuracy, slightly exceeding the full rescue router.

Table 5: Final ablation table.

Router	Accuracy	Gain over baseline	Switch fraction
Oracle accuracy router	0.995875	+0.233187	0.999812
MLP-only rescue	0.765875	+0.003188	0.019563
Full accuracy-rescue router	0.765312	+0.002625	0.021000
MWPM-only rescue	0.764000	+0.001313	0.001687
Empirical MLD-only rescue	0.763938	+0.001250	0.005875
Cheap-only rescue	0.763813	+0.001125	0.002250
Union-find-only rescue	0.763500	+0.000812	0.003812
CNN baseline	0.762687	0.000000	0.000000

This ablation shows that the MLP is the main rescue decoder. The full router remains useful because it demonstrates a general multi-candidate rescue framework, but the strongest

simplified version is the MLP-only rescue gate.

8.2 Random-switch controls

To test whether the learned switch locations matter, random-switch baselines were evaluated at the same overall switch fraction as the learned full rescue router. All random-switch controls were worse than the CNN baseline.

Table 6: Random-switch controls at the same 2.1 percent switch fraction.

Quantity	Value
Mean random-switch accuracy	0.759563
Mean random-switch gain	-0.003125
Best random-switch gain	-0.002562
Full rescue-router gain	+0.002625

The learned full rescue router beat all random-switch controls. This confirms that the gain is not merely due to switching, but due to learned switch placement.

8.3 Recovered oracle gap

The oracle gap over the CNN baseline was

$$0.995875 - 0.762687 = 0.233188.$$

The full learned rescue router recovered

$$\frac{0.002625}{0.233187} \approx 0.0113$$

of this oracle gap.

This is a small fraction, but still meaningful: the router recovers a statistically significant part of a difficult sample-level complementarity signal.

9 Regime-Wise Analysis

The router did not help uniformly across regimes. It improved performance most strongly on easy low-weight, lookup-pattern, and iid-smooth regimes. It was nearly neutral on bursty clusters and slightly harmful on nonlinear and ambiguous mixed regimes.

Table 7: Regime-wise gains of the full rescue router.

Regime	CNN accuracy	Router accuracy	Gain
`easy_low_weight`	0.907295	0.916566	+0.009270
`lookup_pattern`	0.866618	0.872847	+0.006229
`iid_smooth`	0.641335	0.645597	+0.004261
`bursty_cluster`	0.858771	0.859116	+0.000345
`nonlinear_mixed`	0.757296	0.756188	-0.001108
`ambiguous_mixed`	0.524673	0.521299	-0.003374

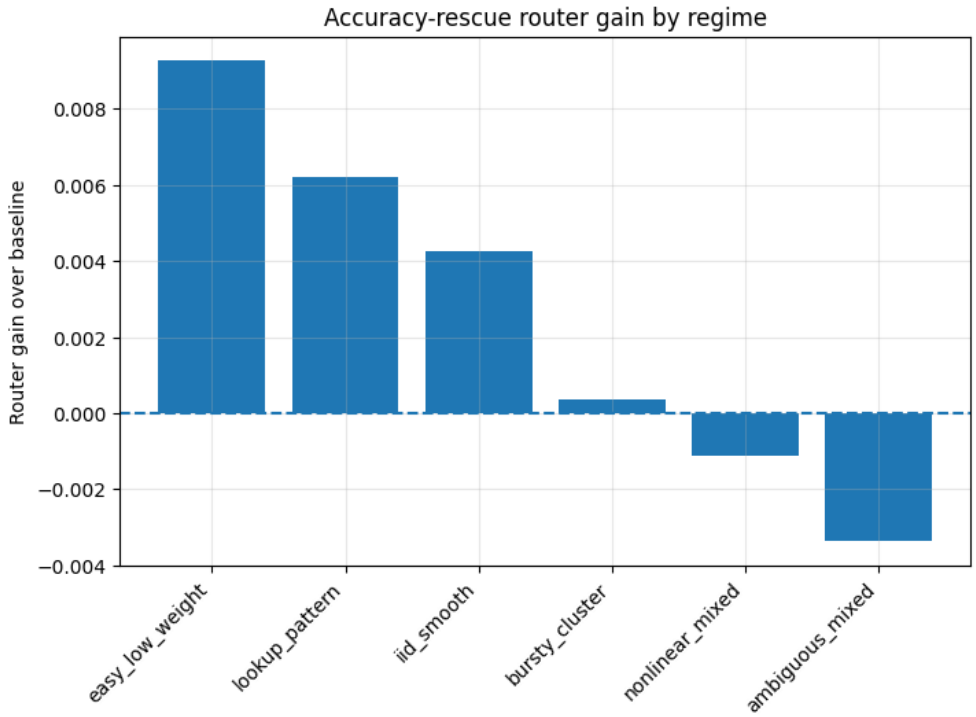

Figure 3: Regime-wise gain of the accuracy-rescue router over the CNN baseline. The router improves most on easy low-weight, lookup-pattern, and iid-smooth regimes, is nearly neutral on bursty clusters, and hurts slightly on nonlinear and ambiguous mixed regimes.

The ambiguous mixed regime remains the most difficult. Its baseline accuracy was low, but the router also struggled there. This suggests that ambiguity alone is insufficient: the router needs observable features that distinguish helpful from harmful switches.

10 Discussion

10.1 What was demonstrated

The main demonstrated result is that, in a structured heterogeneous QEC-inspired benchmark, a learned accuracy-only rescue router can statistically outperform the strongest individual decoder by making rare override decisions.

The result is modest in absolute size but meaningful because:

1. the baseline decoder was already the strongest individual decoder in the portfolio,
2. the router switched on only 2.1 percent of samples,
3. helpful switches outnumbered harmful switches,
4. the bootstrap confidence interval for the full rescue-router gain was positive,
5. random-switch controls at the same switch fraction were consistently worse.

10.2 What was not demonstrated

Several stronger claims are not supported by the present experiment:

- The result does not show that the router beats state-of-the-art QEC decoders.

- The benchmark is QEC-inspired, not a full circuit-level surface-code simulation.
- The learned router recovers only a small fraction of the oracle gap.
- The result has not yet been tested across many independent seeds.
- The current decoder portfolio uses proxy decoders rather than full production-level decoders.

10.3 Why failed routers matter

The failed router attempts were important because they ruled out simpler explanations. Naive multiclass routing failed because selecting the correct decoder directly was too brittle. Utility-aware routing mixed accuracy and compute objectives. Specialist routing failed when the CNN became too dominant or when specialist labels did not align with actual decoder performance. Early rescue routing failed until the gate-training protocol was made held-out and validation-based.

The final method succeeded because it directly optimized the relevant event: candidate correct and baseline wrong, while guarding against the opposite event: candidate wrong and baseline correct.

11 Limitations

The main limitations are:

1. **Synthetic benchmark**: The syndrome generator is structured and QEC-inspired but not a full physical noise model.
2. **Small absolute gain**: The full rescue-router gain is +0.002625, corresponding to 42 additional correct samples out of 16,000.
3. **Large remaining oracle gap**: The oracle accuracy is 0.995875, so most available complementarity remains uncaptured.
4. **Dependence on benchmark design**: Earlier benchmark versions were either too easy or too unlearnable, showing that routing results depend strongly on data generation.
5. **Limited seed analysis**: The final result should be repeated over multiple seeds before making stronger claims.
6. **Proxy decoder portfolio**: MWPM-like and union-find-like decoders are simplified proxies.

12 Future Work

12.1 Multi-seed validation

The immediate next step is to repeat the final experiment across multiple seeds. The desired result is a positive mean gain with most seeds showing positive improvement.

12.2 Simplified MLP-only rescue router

Since the MLP-only rescue router outperformed the full rescue router, a simplified final method may be preferable: default to CNN and switch to MLP only when the MLP rescue gate fires. This simpler router is easier to explain and may generalize better.

12.3 Stim and PyMatching benchmark

A more realistic next phase is to construct a circuit-level benchmark using Stim for sampling detector events and PyMatching as a strong MWPM baseline [3, 2]. A realistic portfolio could include PyMatching MWPM, a neural decoder, a lookup decoder for low-distance patterns, a cluster or union-find-inspired decoder, and a learned rescue router. This would move the project from a structured proof-of-concept to a more physically meaningful QEC decoding study.

12.4 Compute-aware routing

The present report focused on accuracy only. However, earlier utility-oracle diagnostics showed that cheap decoders can be correct on many samples. A future compute-aware router could aim to maintain near-CNN accuracy while reducing average compute by using cheap decoders when they are safe.

13 Conclusion

This project began with the hypothesis that a decoder portfolio may contain exploitable complementarity beyond the best individual decoder. Initial experiments confirmed large oracle complementarity but also showed that naive learned routers could not reliably exploit it. Several failed router designs led to a clearer formulation: use the strongest decoder as a default and learn rare rescue gates for candidate decoders.

The final accuracy-only rescue router achieved a statistically meaningful improvement over the CNN baseline on a large structured heterogeneous benchmark. The CNN baseline achieved accuracy 0.762687, while the full rescue router achieved 0.765312, with bootstrap 95 percent confidence interval [+0.000375,+0.004938]. The gain came from sparse switching: only 2.1 percent of samples were rerouted, with 189 helpful and 147 harmful switches. The MLP-only rescue ablation achieved an even higher accuracy of 0.765875, showing that most of the learned gain came from rare CNN-to-MLP rescue cases.

The central conclusion is modest but meaningful: learned rescue routing can recover a statistically significant part of decoder complementarity in a controlled QEC-inspired benchmark. The result motivates future work on realistic circuit-level benchmarks and stronger decoder portfolios.

A Development Timeline

A.1 Stage 1: Initial oracle complementarity

The project began by testing whether a portfolio of decoders contained more information than any one decoder. Oracle accuracy was much higher than the best individual decoder, suggesting substantial complementarity.

A.2 Stage 2: Naive learned routers

Several multiclass routers were trained to predict the best decoder. These failed because they switched too often and accumulated harmful switches.

A.3 Stage 3: Residual rescue routers

Residual routers defaulted to the best decoder and attempted to switch only on predicted failures. Early small-run gains did not survive scaling.

A.4 Stage 4: Learnability audit

Diagnostics separated oracle complementarity from learnable complementarity. Some benchmark versions had high oracle accuracy but no regime-level routing signal.

A.5 Stage 5: Structured benchmark redesign

The benchmark was redesigned to include explicit heterogeneous regimes. Some versions were too easy, allowing CNN domination, while others were too noisy or mixed. The final harder benchmark produced enough headroom for routing.

A.6 Stage 6: Held-out accuracy-rescue routing

The final successful router trained rescue and harm gates on held-out decoder behavior and tuned thresholds on a separate validation subset. This avoided misleading training-split behavior from memorization-style decoders.

B Key Numerical Results

Table 8: Key final numbers.

Quantity	Value
Best individual decoder	CNN neural decoder
Best individual accuracy	0.762687
Oracle accuracy	0.995875
Full rescue-router accuracy	0.765312
Full rescue-router gain	+0.002625
MLP-only rescue accuracy	0.765875
MLP-only rescue gain	+0.003188
Bootstrap CI, full router	[+0.000375, +0.004938]
Probability of positive gain, full router	0.9864
Switch fraction, full router	0.021
Helpful switches	189
Harmful switches	147
Net correct samples	+42
Recovered oracle gap fraction	0.0113

References

[1] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, "Topological quantum memory," *Journal of Mathematical Physics*, vol. 43, pp. 4452–4505, 2002.

[2] O. Higgott, "PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching," *ACM Transactions on Quantum Computing*, vol. 3, no. 3, pp. 1–16, 2022.

[3] C. Gidney, "Stim: a fast stabilizer circuit simulator," *Quantum*, vol. 5, p. 497, 2021.

[4] N. Delfosse and N. H. Nickerson, "Almost-linear time decoding algorithm for topological codes," *Quantum*, vol. 5, p. 595, 2021.